本轮进展：减少整次miss与重复计划

上轮主要压缩单个函数的内部开销。本轮进一步让当前miss与已知下一目标共用容量计划，并解决联合计划触发完整排序所带来的CPU增加。最终提供保守自适应版与保持原动作的双候选快速版。

500万操作混合输入 / 各3次中位数	上一轮基础实现	本轮最终实现
实际miss	1,469,461	1,180,392
决策时间 / 微秒	3,227,784.63	2,973,349.02
完整Score	5,395,769,115.45	5,103,835,083.08
墙上时间	4.92秒	4.67秒

这个压力场景中，完整Score降低5.41%，决策时间降低7.88%。数据来自本机已知规模回归，不是官方隐藏测试。

通用验证没有被省略

第一批576个新输入显示，不加限制的联合策略略有退化。随后加入保守门槛，再冻结并生成另一批300例。后者整体表现接近原实现，不能由单个大输入的收益宣布全分布领先。

自主贡献

相邻目标一次规划；对排序前两个候选足够的情况进行精确O(Q)化简；把新策略避免的miss加回压力反馈，避免控制器因预取成功而误判内存压力消失。这些是本题受限接口下的具体实现与论证，不声称全球首次或全局最优。

一次计划，处理当前与下一目标

原行为与新行为

原实现先为当前目标A规划空间，只有剩余空间恰好足够时才顺便加载下一目标B。若不够，B通常还要发生一次miss、构造状态快照并重新选择victim。

新实现知道next_op明确为PROBE B，且A、B都在磁盘、都处于PROBE、可共同容纳时，尝试直接为size(A)+size(B)规划空间。返回SPILL集合、LOAD B、LOAD A；最后一条仍为LOAD当前目标A。

状态例：M=64MiB，驻留8个8MiB分区，A/B各8MiB在磁盘	操作
分别处理	A的miss释放8MiB并读A；B的miss再释放8MiB并读B
联合处理	A的miss一次释放16MiB；先读B、再读A；随后访问B命中
这个例子的代价	都写16MiB、读16MiB；联合方案有机会减少一次miss和一份重复计划

例子假设所选victim及后续访问使两种方案的I/O相同；实际负载的选择会受访问历史影响。联合保留A/B也可能挤掉未来更需要的分区，因此必须有保护门槛。

合法信息边界

只使用当前状态与紧邻的一条next_op；没有读取B之后的操作C，没有预读输入文件或访问op_lines。B可以来自其他算子，所有动作仍受全局M约束。

题面2.5给出next_op和动作列表；公开引擎允许在当前miss的动作中LOAD其他PROBE分区，并要求末尾LOAD当前目标。本轮保持引擎原样，仅替换调度器。

官方第20题：接口与动作约束

自适应启用，限制联合加载的代价

保守版同时要求以下条件

条件	用途
当前在线规则已经切换为观测热度/大小	利用运行中的读回反馈识别持续换入压力；首次miss不直接启用联合模式
A+B不超过M/2	给其余工作集保留容量空间，避免两个目标占满整个缓存
A/B大小相差不超过2倍	集中处理规模相近的请求，减少为一个小请求牵动大分区
每个victim不大于较大目标加容差	保护可能很重要的大驻留分区
联合写出总量不超过A+B加容差	限制容量贪心的写出超额
容差=max(16字节,(A+B)/256)	容纳字节级边界变化；不是隐藏罚时或外部参数

不符合门槛时，沿用单目标调度与原有低成本预取。规则本身仍随阶段、窗口内压力和短间隔读回变化；没有从文件名、N、随机种子或预设赛题类型识别输入。

大热分区反例

M=64MiB，驻留一个56MiB热分区和一个8MiB小分区；磁盘A/B各8MiB。保留A+B可能迫使56MiB热分区落盘，而逐个处理可以轮换8MiB小分区。本轮先构造此类输入，再加入大分区保护与总写量门槛。

这些条件降低已见风险，不是竞争比保证。未知C仍可能使提前保留A/B变差；完整结果和最差案例均保留。

混合BUILD/PROBE阶段若额外淘汰了BUILD分区，还可能影响其后续追加写盘成本；本轮门槛没有给出这部分的全局风险界。

修正反馈：预取成功不等于压力消失

直接统计实际miss会产生内生偏差

联合加载减少B的实际miss，也使B的miss回调消失。如果控制器只观察实际miss率，可能把“提前处理了需求”误认为“不再需要换入”，随后过早切回不合适的规则。

保守版的修正量

有效压力计数 = 实际miss数 + 新联合计划避免的miss数。全局Probe时钟仍使用hit_count+miss_count，不修改题目状态或实际Score。

如何判断是否确实避免了一次miss

联合只在热度/大小规则下启用，该规则的victim是排序前缀。对联合计划的SPILL序列，从前往后累计，找到只满足A需求所需的最短前缀；如果这个前缀留下的空间仍装不下B，则原单目标计划不会免费预取B。

在保守门槛下，B<M/2；原周期性强制预取分支也不会为这样的B增加落盘。因此从同一个当前状态和规则出发，B原本会产生miss，可以加回1。若B原本就能免费预取，则不加回。

证明范围

该重建比较同一个当前状态、同一个热度规则的单目标前缀，不等于模拟了一条完整反事实基线。未来驻留状态和规则仍可能分叉，修正用于控制器学习压力，不构成整段Score无遗憾保证。

相比把所有LOAD都当作压力事件，这个版本避免把原有免费预取再次收费。它使用自己的计划与历史，不读取真实miss_penalty。代价策略里的32仍是既有固定尺度。

双候选快速路径：保持原动作，省去完整排序

触发条件

最优排序候选单独不够，但按原比较器最前面的两个候选合计足够覆盖缺口。此时原贪心排序前缀长度恰为2，可以用两次线性选择直接找出这两个候选。

必须保留的细节

代价规则仍扫描所有能独立覆盖缺口的候选，比较其总代价；若有更好的单分区方案仍选择它。之后按原顺序去掉多余victim。字节大小与ID平局规则保持一致。两个候选不够时，继续使用完整排序。

等价性论证

原比较器按key、大小、ID定义全序。线性选出的第一、第二项与全排序前两项相同；覆盖条件使原前缀恰为这两项。总代价之和的顺序不变；单覆盖比较与冗余删除不变，因此返回的SPILL顺序完全相同。

这证明与原贪心启发式等价，不是证明求出了最优背包覆盖。快速分支O(Q)，回退仍为O(Q log Q)，Q<=512。没有近似倒数、fast-math或随机截断。

相同工作量的计时消融

此项消融固定使用较宽联合策略（配置41），两列只改变排序实现；它与主候选配置105的miss数不同，不能把两项试验混成同一配置。

500万混合输入 / 各3次	完整排序联合版	双候选联合版
miss	848,739	848,739
I/O时间	3,921,126,355.55	3,921,126,355.55
决策时间 / 微秒	5,178,473.14	2,476,254.01

这组相同I/O、相同miss的对照中，决策时间降低52.18%。它解释了为何只做联合加载会增加CPU，而补上结构化快速路径才能兑现收益。

两批新输入：保留没有改善的结果

第一阶段：576例，各运行3次。组成是20类分布320例、128参数端点、6类针对场景96例和32个新大热分区反例。源文件先冻结再抽系统熵。

第一批 / 正BASE例	几何VAL/BASE	公式均值/10	miss
基础版	0.588345	2.4079	157,072
较宽联合版	0.589114	2.4003	151,883

较宽版本减少miss，但整体几何比与公式均值略差。不能把开发集或规模输入的改善当成通用改善。第一批随后成为收紧门槛的开发证据。

第二阶段：保守策略冻结后另生成300例，各运行3次。组成是20类分布120例、128端点、6类针对场景36例和16个新反例。

第二批 / 300例	几何VAL/BASE	公式均值/10	累计决策us
基础版	0.676539	2.1514	229,186.58
纯双候选快速版	0.676827	2.1528	218,738.90
保守联合版	0.676430	2.1509	218,777.91

300例是策略的独立新输入；轻量入口复用了这批数据做代码等价和计时复测，没有再把它称为第三批独立数据。两批的分布与种子不同，数值不能跨批直接相减。

BASE为0的输入保留合法性检查，但不参与比值汇总。公式均值只是按单组满分10计算，没有转换为官方总分。通用验证整体接近，尚无跨分布全面领先证据。

三轮规模验证：收益来自哪些成本

每个配置、每个输入重复3次，交替顺序，表中取中位数。百分比相对上一轮基础实现，负值表示退化；Score包含I/O、真实输入罚时和全部决策时间。

场景	决策耗时降低	完整Score降低	原miss / 新miss
spilled_hot_B1	-2.94%	-2.86647%	0 / 0
resident_pressure_B1	4.97%	0.54120%	22,088 / 18,543
resident_pressure_B1000	4.21%	0.62880%	21,869 / 18,401
maxN_maxQ_B1_hits	-1.83%	-1.83458%	0 / 0
maxN_mixed_maxQ	7.88%	5.41042%	1,469,461 / 1,180,392
maxN_growth_4GiB	7.00%	7.73188%	187 / 136

回调数核对

500万混合输入	原基础版	保守联合核
总回调	1,474,717	1,185,648
miss回调	1,469,461	1,180,392
周期回调	4,999	4,999
双候选快速分支	原实现无此分支	288,070

计数来自独立严格回放，时间来自原样公开引擎。周期次数由B决定，未被修改；减少的是实际miss触发的回调。I/O时间是题目模拟值，墙上时间是程序实跑时长，二者不能混为实际磁盘等待。

被拒绝的轻量入口，以及无法消除的固定成本

轻量入口不改动作

完整调度核仍在构造阶段初始化。尚未发生落盘时，周期调用只检查下一INSERT是否真的需要腾空间；其他情况直接返回空动作。出现overflow、miss或可预测的溢出后，后续调用转交完整核。

原核在首次落盘前同样不会维护热度或返回其他动作，所以这段短入口保留行为。独立300例逐动作对照验证其与保守调度核的动作、读写字节、hit/miss和回调数一致。

额外成本也计入

入口增加了一个标准库指针拥有的调度对象与活跃标志；激活后有一次内部转交。没有把状态快照移出计时区间，也没有把在线推断搬到未计时线程。构造阶段仍只做原有核初始化，新增一个对象分配。

复测中，入口版的累计CPU与规模结果没有稳定优于直接调度核，故没有作为最终主文件；源码与等价审计保留为失败候选。

框架固定开销仍存在

上轮采样显示空回调内部约20~30ns，而包含SystemState复制等部分的完整回调约0.28微秒。入口优化无法消除框架成本；因此联合处理减少整次miss回调比继续缩短空函数更有价值。

CPU与信息之间的tradeoff

提前加载B会失去B真实miss时才能看到的下一操作C；新策略用容量、尺寸和写量门槛限制这笔信息代价。另一方面，复杂联合计划若总是排序，会抵消减少回调的收益；双候选化简处理了这部分计算成本。

读写带宽、目标大小、驻留集、反馈损失与动态规则都来自合法状态。真实miss_penalty仍不可见，代码没有读取输入、N、B或更远未来。

交付、验证与当前使用建议

当前候选

文件	用途
MySpillScheduler_final.cpp	保守自适应联合 + 双候选快速路径，直接使用调度核；标准库C++14，默认无参数构造
MySpillScheduler_prefix2.cpp	只加入精确双候选快速路径；保持原基础策略的全部动作，适合优先控制策略变化
MySpillScheduler_joint.cpp	较宽联合研究版；压力场景收益更大，但第一批随机验证略有退化，不作为通用默认
next_probe_20261026/README.txt	逐步复现、参数门槛、源文件冻结、原始Score与反例说明

不要把这些同名类文件同时编译；选择一个调度器文件替换即可。此前默认提交未被覆盖。

核验

标准为C++14，源码只使用标准库；无main、外部模型、输入读取或隐藏罚时。公开引擎仅作允许的调度器替换。两阶段源文件与876个新输入摘要均核验。

双候选快速路径与慢排序版的动作/I/O等价；纯快速基础版与原基础版等价；轻量入口与保守调度核等价。新联合策略改变动作，按独立合法性、容量与真实Score评估。所有规模运行低于60秒。

当前判断

更明显的收益集中在频繁换入、目标大小接近的压力场景。随机新输入总体接近原策略，仍存在退化与计时波动。可以把保守版作为下一轮候选；不能由本机合成负载保证官方成绩、所有输入收益或全局最优。

下一轮若继续，应优先研究联合加载的未来信息损失及更可靠的局部收益估计；本轮没有用夸大的平均数掩盖这些未解决问题。